

```

from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error

```

```
data_set= load_diabetes()
```

```
df = pd.DataFrame(data_set.data)
```

```
df
```



	0	1	2	3	4	5	6	7	8	9
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019907	-0.017646
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493	-0.068332	-0.092204
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	-0.002592	0.002861	-0.025930
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309	0.022688	-0.009362
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031988	-0.046641
...	...	...	...	...	...	...	...	...	...	...
437	0.041708	0.050680	0.019662	0.059744	-0.005697	-0.002566	-0.028674	-0.002592	0.031193	0.007207
438	-0.005515	0.050680	-0.015906	-0.067642	0.049341	0.079165	-0.028674	0.034309	-0.018114	0.044485
439	0.041708	0.050680	-0.015906	0.017293	-0.037344	-0.013840	-0.024993	-0.011080	-0.046883	0.015491
440	-0.045472	-0.044642	0.039062	0.001215	0.016318	0.015283	-0.028674	0.026560	0.044529	-0.025930
441	-0.045472	-0.044642	-0.073030	-0.081413	0.083740	0.027809	0.173816	-0.039493	-0.004222	0.003064

442 rows × 10 columns

```

n_df = pd.DataFrame({f'feature{i+1}': df[i] for i in range(10)})
display(n_df)

```



	feature1	feature2	feature3	feature4	feature5	feature6	feature7	feature8	feature9	feature10
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019907	-0.017646
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493	-0.068332	-0.092204
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	-0.002592	0.002861	-0.025930
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309	0.022688	-0.009362
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031988	-0.046641
...	...	...	...	...	...	...	...	...	...	...
437	0.041708	0.050680	0.019662	0.059744	-0.005697	-0.002566	-0.028674	-0.002592	0.031193	0.007207
438	-0.005515	0.050680	-0.015906	-0.067642	0.049341	0.079165	-0.028674	0.034309	-0.018114	0.044485
439	0.041708	0.050680	-0.015906	0.017293	-0.037344	-0.013840	-0.024993	-0.011080	-0.046883	0.015491
440	-0.045472	-0.044642	0.039062	0.001215	0.016318	0.015283	-0.028674	0.026560	0.044529	-0.025930
441	-0.045472	-0.044642	-0.073030	-0.081413	0.083740	0.027809	0.173816	-0.039493	-0.004222	0.003064

442 rows × 10 columns

```
n_df.rename(columns={'feature10': 'target'}, inplace=True)
```

```
n_df
```



	feature1	feature2	feature3	feature4	feature5	feature6	feature7	feature8	feature9	target
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019907	-0.017646
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493	-0.068332	-0.092204
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	-0.002592	0.002861	-0.025930
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309	0.022688	-0.009362
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031988	-0.046641
...	...	...	...	...	...	...	...	...	...	...
437	0.041708	0.050680	0.019662	0.059744	-0.005697	-0.002566	-0.028674	-0.002592	0.031193	0.007207
438	-0.005515	0.050680	-0.015906	-0.067642	0.049341	0.079165	-0.028674	0.034309	-0.018114	0.044485
439	0.041708	0.050680	-0.015906	0.017293	-0.037344	-0.013840	-0.024993	-0.011080	-0.046883	0.015491
440	-0.045472	-0.044642	0.039062	0.001215	0.016318	0.015283	-0.028674	0.026560	0.044529	-0.025930
441	-0.045472	-0.044642	-0.073030	-0.081413	0.083740	0.027809	0.173816	-0.039493	-0.004222	0.003064

442 rows × 10 columns

n_df.shape



(442, 10)

```
X = n_df[["feature{i}" for i in range(1, 10)]]
Y = n_df['target']
```

X



	feature1	feature2	feature3	feature4	feature5	feature6	feature7	feature8	feature9
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019907
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493	-0.068332
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	-0.002592	0.002861
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309	0.022688
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031988
...	...	...	...	...	...	...	...	...	...
437	0.041708	0.050680	0.019662	0.059744	-0.005697	-0.002566	-0.028674	-0.002592	0.031193
438	-0.005515	0.050680	-0.015906	-0.067642	0.049341	0.079165	-0.028674	0.034309	-0.018114
439	0.041708	0.050680	-0.015906	0.017293	-0.037344	-0.013840	-0.024993	-0.011080	-0.046883
440	-0.045472	-0.044642	0.039062	0.001215	0.016318	0.015283	-0.028674	0.026560	0.044529
441	-0.045472	-0.044642	-0.073030	-0.081413	0.083740	0.027809	0.173816	-0.039493	-0.004222

442 rows × 9 columns

Y



	target
0	-0.017646
1	-0.092204
2	-0.025930
3	-0.009362
4	-0.046641
...	...
437	0.007207
438	0.044485
439	0.015491
440	-0.025930
441	0.003064

442 rows × 1 columns

dtype: float64

```
X_train, X_test, y_train, y_test = train_test_split(
    X, Y, test_size=0.2, random_state=42)
```

```
model = LinearRegression()
```

```
model.fit(X_train,y_train)
```

```
LinearRegression
```

```
predictions = model.predict(X_test)
```

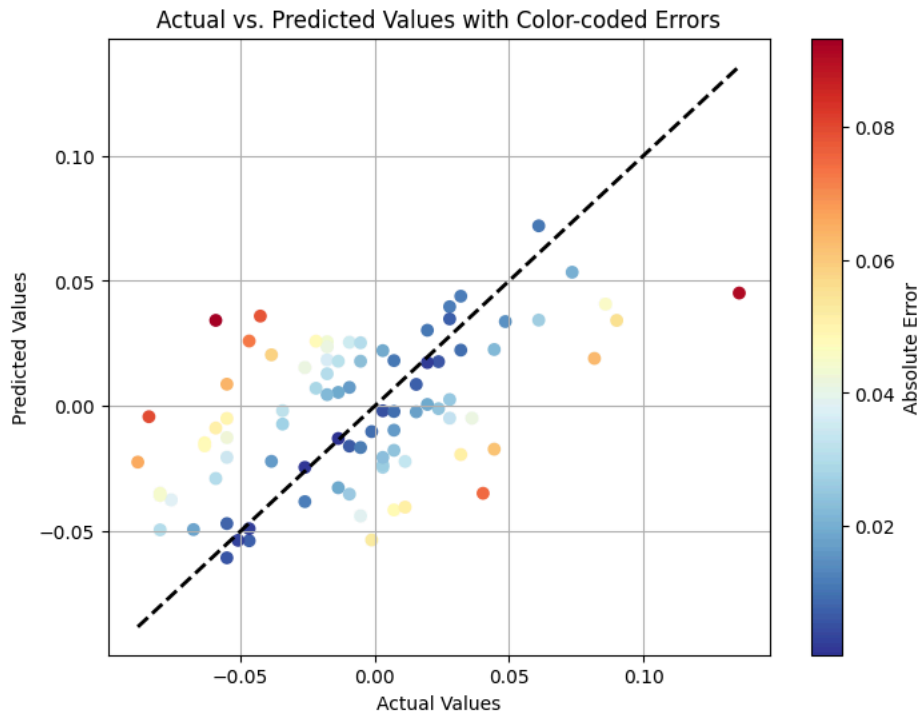
```
predictions
```

```
array([[ 0.01795242,  0.00714908,  0.02570788,  0.07197301,  0.00749053,
        -0.00957196,  0.04389301,  0.03435451, -0.02439529, -0.01014245,
        -0.02200251, -0.00419319, -0.06050463,  0.03423753, -0.01715733,
        -0.01930115,  0.04074458,  0.05346341, -0.00226367,  0.03969703,
         0.00552473, -0.0146184 , -0.04140273,  0.03030806, -0.01592567,
         0.00870107,  0.01847317,  0.01795839, -0.05369374, -0.00098385,
         0.00069896, -0.0380798 , -0.00184815,  0.02238674,  0.0177745 ,
         0.02550624, -0.00476394,  0.00263898,  0.01907617, -0.03739968,
        -0.03476065, -0.00211286,  0.00877726,  0.02261898,  0.01819353,
        -0.03476505, -0.0533734 , -0.02339817, -0.04889181, -0.00488207,
        -0.02440359, -0.04935076, -0.00871194, -0.02043729,  0.03596397,
        -0.01583155, -0.02205483,  0.02369848, -0.02041138, -0.04381903,
         0.0221781 ,  0.00461399, -0.01650421, -0.0047823 , -0.01288328,
         0.03425578,  0.02530696,  0.0204953 , -0.0355862 , -0.00178393,
         0.0129312 ,  0.03482119,  0.04515843, -0.00713976, -0.02884561,
         0.02600916,  0.03376684,  0.01543325,  0.01740668,  0.02593953,
        -0.03254013, -0.02231277, -0.04684546, -0.03509398, -0.01761152,
        -0.04027806, -0.04923399, -0.05353426, -0.0125186 ]])
```

```
from sklearn.metrics import r2_score
from sklearn.metrics import accuracy_score
print(
    'MSE : ', mean_squared_error(y_test, predictions))
print(
    'MAE : ', mean_absolute_error(y_test, predictions))
print('R-S: ', r2_score(y_test, predictions))
```

```
MSE : 0.0014397511613178668
MAE : 0.031211788744898387
R-S: 0.24545282143100922
```

```
plt.figure(figsize=(8, 6))
plt.scatter(y_test, predictions, c=np.abs(y_test - predictions), cmap='RdYlBu_r')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'k--', lw=2)
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.title("Actual vs. Predicted Values with Color-coded Errors")
plt.colorbar(label="Absolute Error")
plt.grid(True)
plt.show()
```



```
from sklearn.datasets import fetch_california_housing
housing = fetch_california_housing()
```

```
housing_df = pd.DataFrame(housing.data, columns=housing.feature_names)
housing_df['target'] = housing.target
```

```
housing_df
```



	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	target
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422
...	...	...	...	...	...	...	...	...	...
20635	1.5603	25.0	5.045455	1.133333	845.0	2.560606	39.48	-121.09	0.781
20636	2.5568	18.0	6.114035	1.315789	356.0	3.122807	39.49	-121.21	0.771
20637	1.7000	17.0	5.205543	1.120092	1007.0	2.325635	39.43	-121.22	0.923
20638	1.8672	18.0	5.329513	1.171920	741.0	2.123209	39.43	-121.32	0.847
20639	2.3886	16.0	5.254717	1.162264	1387.0	2.616981	39.37	-121.24	0.894

20640 rows × 9 columns

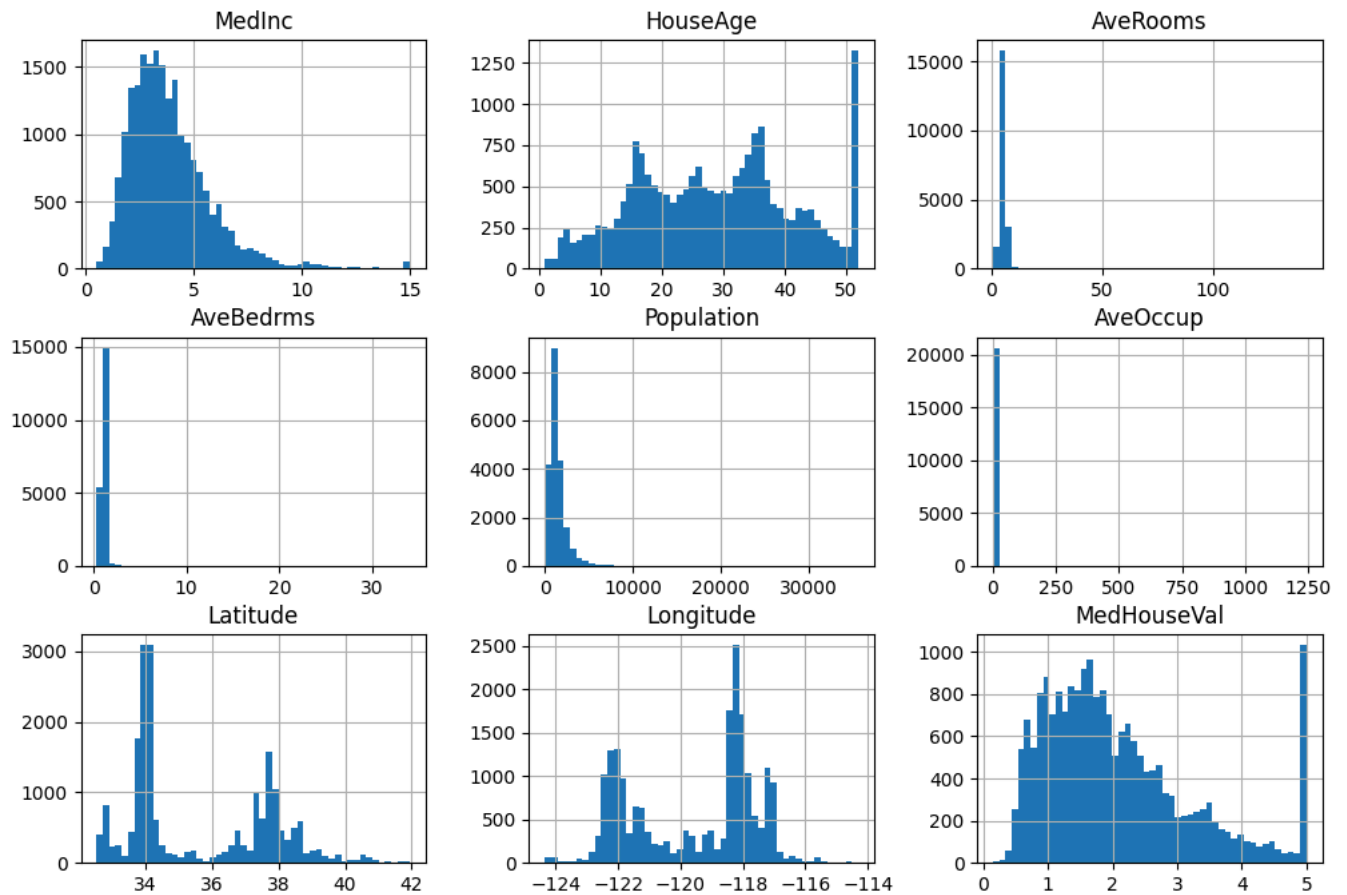
```
import pandas as pd
import numpy as np
from sklearn.datasets import fetch_california_housing
import matplotlib.pyplot as plt
from pandas.plotting import scatter_matrix
from sklearn.metrics import r2_score
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
```

```
housing = fetch_california_housing(as_frame=True)
housing = housing.frame
housing.head()
```

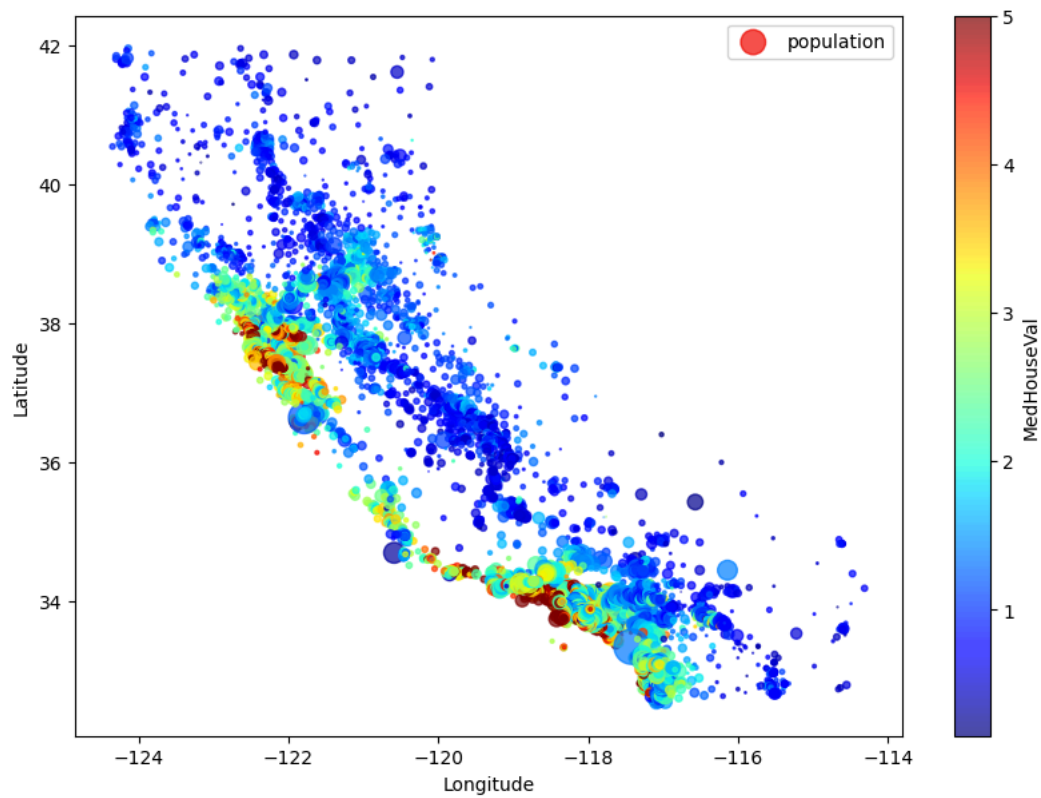


	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422

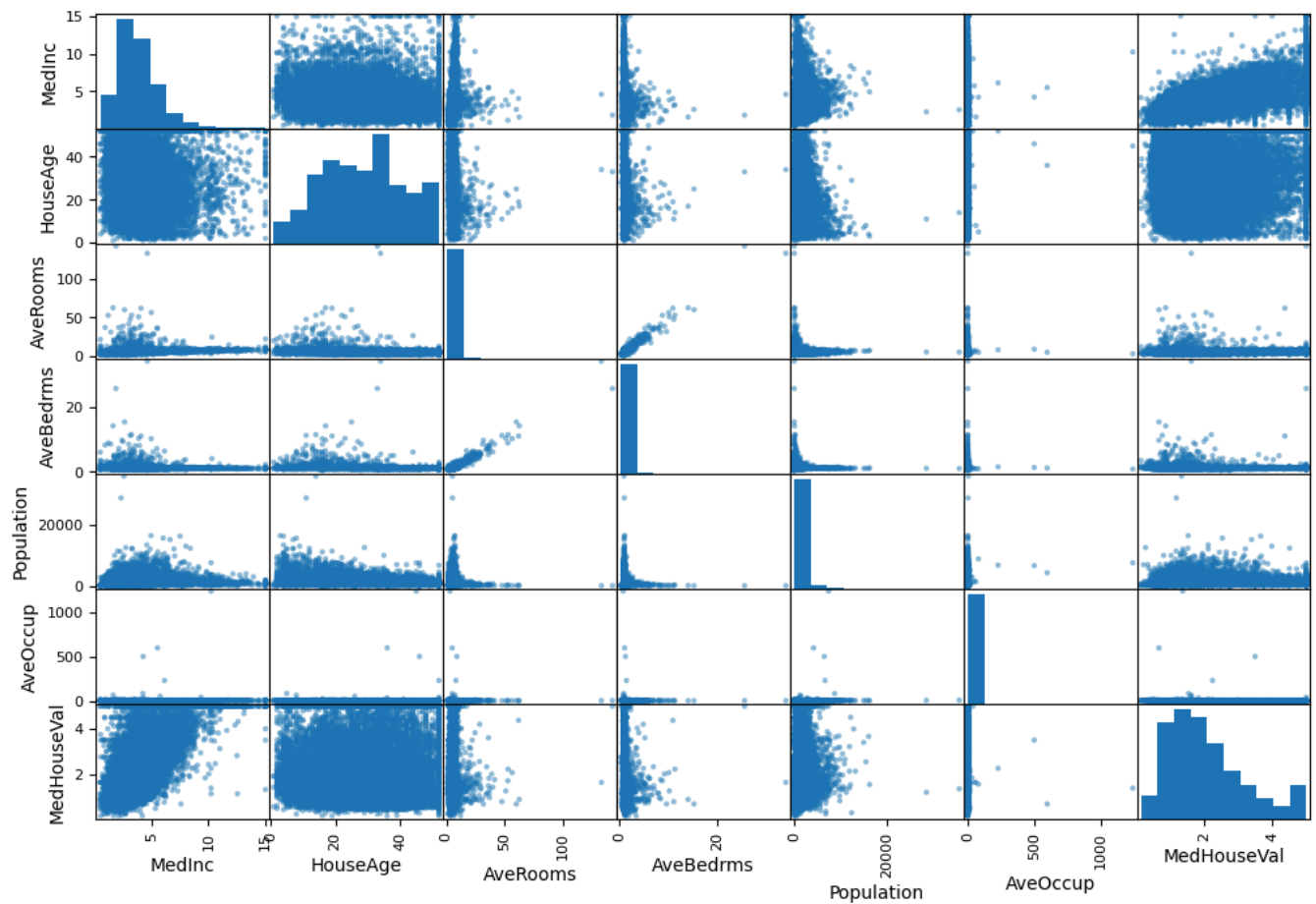
```
housing.hist(bins=50, figsize=(12,8))
plt.show()
```



```
housing.plot(kind="scatter", x="Longitude",y="Latitude", c="MedHouseVal", cmap="jet", colorbar=True, legend=True, sharex=False, figsize=
plt.show())
```



```
attributes = ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup', 'MedHouseVal']
scatter_matrix(housing[attributes], figsize=(12,8))
plt.show()
```



```
corr = housing.corr()
corr['MedHouseVal'].sort_values(ascending=True)
```



	MedHouseVal
Latitude	-0.144160
AveBedrms	-0.046701
Longitude	-0.045967
Population	-0.024650
AveOccup	-0.023737
HouseAge	0.105623
AveRooms	0.151948
MedInc	0.688075
MedHouseVal	1.000000

housing.data() sum()

Q1 Is a multivariate regression the same as linear regression?

No, multivariate regression is not the same as linear regression.

Linear regression involves one independent and one dependent variable. In contrast, multivariate regression uses two or more independent variables, and sometimes multiple dependent variables, to study more complex relationships.

Q2 What is a multivariate regression with example?

Multivariate regression is a method used to model the relationship between multiple inputs and multiple outputs. For example, it can be used to predict both blood pressure and cholesterol level based on variables like age, weight, and diet. It's helpful in analyzing systems with multiple influencing

Q3 Why do we use multivariate regression models?

We use multivariate regression to understand how several variables affect one or more outcomes. It allows researchers to model real-world situations more accurately and uncover relationships that wouldn't be visible with simpler models. It's especially useful when outcomes are influenced by many factors.

Q4 What are the advantages of multivariate regression?

It offers improved prediction accuracy and reveals complex variable interactions. It allows control over confounding variables and helps researchers make more reliable and generalizable conclusions. It's also efficient, as it analyzes multiple variables in a single model.

Q5 Is multivariable regression linear regression?

Yes, multivariable regression is a type of linear regression that involves more than one independent variable predicting a single dependent variable. Although the number of predictors increases, the relationship between variables remains linear. It's an extension of simple linear regression.

Q6 What is the difference between univariate and multivariate regression?

Univariate regression involves only one dependent variable and usually one independent variable.

Multivariate regression, on the other hand, can include multiple independent and/or dependent variables. It provides a deeper understanding of how different factors influence outcomes together.

Q7 Where is multivariate analysis put to use?

Multivariate analysis is used in many fields like marketing, education, healthcare, and social sciences. It helps in understanding how multiple factors work together to influence outcomes. For example, in marketing, it can analyze customer behavior based on age, income, and preferences.

Q8 Is multivariate analysis qualitative or quantitative?

Multivariate analysis is primarily quantitative, as it often deals with numerical data. However, it can also be applied to qualitative or categorical data using techniques like logistic regression. It's flexible enough to handle both types depending on the method used.